NAME:ANANYA MANISH JADHAV

CLASS:A

BATCH:A4

PRN :202401040097

ROLL NO:62

CODE

```
#include <iostream>
#include <stack>
#include <queue>
using namespace std;
struct Node {
   int data;
   Node* left;
   Node* right;
   Node(int v) {
      data = v;
      left = right = NULL;
   }
};

Node* insertNode(Node* root, int v) {
   if (root == NULL) return new Node(v);

   if (v < root->data)
      root->left = insertNode(root->left, v);
```

```
    else if (v > root->data)
        root->right = insertNode(root->right, v);

    return root;
}

Node* findMin(Node* root) {
    while (root && root->left != NULL)
        root = root->left;
    return root;
}

Node* deleteNode(Node* root, int key) {
    if (root == NULL) return root;
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Case 1: Leaf
        if (root->left == NULL && root->right == NULL) {
            delete root;
            return NULL;
        }
        // Case 2: One child
        else if (root->left == NULL) {
```

```
        Node* temp = root->right;
        delete root;
        return temp;
      }
      else if (root->right == NULL) {
        Node* temp = root->left;
        delete root;
        return temp;
      }
      // Case 3: Two children
      Node* temp = findMin(root->right);
      root->data = temp->data;
      root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
void preorder(Node* root) {
    if (!root) return;
    cout << root->data << " ";
```

```
    preorder(root->left);
    preorder(root->right);
}
void postorder(Node* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
void inorderNR(Node* root) {
    stack<Node*> st;
    Node* curr = root;
    while (curr || !st.empty()) {
        while (curr) {
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top(); st.pop();
        cout << curr->data << " ";
        curr = curr->right;
    }
}
void preorderNR(Node* root) {
    if (!root) return;
    stack<Node*> st;
    st.push(root);
```

```
    while (!st.empty()) {
        Node* curr = st.top(); st.pop();
        cout << curr->data << " ";
        if (curr->right) st.push(curr->right);
        if (curr->left) st.push(curr->left);
    }
}
void postorderNR(Node* root) {
    if (!root) return;
    stack<Node*> s1, s2;
    s1.push(root);
    while (!s1.empty()) {
        Node* curr = s1.top(); s1.pop();
        s2.push(curr);
        if (curr->left) s1.push(curr->left);
        if (curr->right) s1.push(curr->right);
    }

    while (!s2.empty()) {
        cout << s2.top()->data << " ";
        s2.pop();
    }
}
void levelOrder(Node* root) {
    if (!root) return;
    queue<Node*> q;
```

```
    q.push(root);
    while (!q.empty()) {
        Node* curr = q.front(); q.pop();
        cout << curr->data << " ";
        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}
int main() {
    Node* root = NULL;
    int ch, val;
    do {
        cout << "\n--- BST MENU ---\n";
        cout << "1.Insert\n2.Delete\n3.Inorder(R)\n4.Preorder(R)\n5.Postorder(R)\n";
        cout << "6.Inorder(NR)\n7.Preorder(NR)\n8.Postorder(NR)\n9.LevelOrder\n0.Exit\n";
        cout << "Choice: ";
        cin >> ch;
        switch (ch) {
        case 1:
            cout << "Enter value: ";
            cin >> val;
            root = insertNode(root, val);
            break;
        case 2:
            cout << "Enter value to delete: ";
            cin >> val;
```

```
        root = deleteNode(root, val);
        break;
    case 3:
        inorder(root);
        break;
    case 4:
        preorder(root);
        break;
    case 5:
        postorder(root);
        break;
    case 6:
        inorderNR(root);
        break;
    case 7:
        preorderNR(root);
        break;
    case 8:
        postorderNR(root);
        break;
    case 9:
        levelOrder(root);
        break;
    }
    cout << endl;
} while (ch != 0);
```

```
    return 0;
}
```

OUTPUT:

BST MENU

1.Insert

2.Delete

3.Inorder(R)

4.Preorder(R)

5.Postorder(R)

6.Inorder(NR)

7.Preorder(NR)

8.Postorder(NR)

9.LevelOrder

0.Exit

Choice: 1

Enter value: 50

Choice: 1

Enter value: 30

Choice: 1

Enter value: 70

Choice: 1

Enter value: 20

Choice: 1

Enter value: 40

Choice: 1

Enter value: 60

Choice: 1

Enter value: 80

Choice: 3

Inorder Traversal:

20 30 40 50 60 70 80

Choice: 4

Preorder Traversal:

50 30 20 40 70 60 80

Choice: 5

Postorder Traversal:

20 40 30 60 80 70 50

Choice: 9

Level Order Traversal:

50 30 70 20 40 60 80

Choice: 2

Enter value to delete: 30

Choice: 3

Inorder Traversal After Delete:

20 40 50 60 70 80

Choice: 0